

Conversational AI Semantic Search for E-commerce using Elasticsearch + RAG: A Scalable Hybrid Framework

Karthik Govardhanan
Boston University
233 Bay State Road
Boston, MA-02215, USA
kartigk@bu.edu

Abstract—This work provides a modular hybrid information retrieval architecture that combines Elasticsearch with RAG to send context-aware, accurate, and low-latency replies to user queries. The system has four primary modules: transformer-based reformulation for query comprehension, BM25 and k-NN embeddings for lexical and dense retrieval, transformer models for generative answer synthesis, and a ranking-feedback loop for adaptive optimisation. In a simulated retail environment with synthetic product data and queries, the hybrid system outperforms standalone Elasticsearch and RAG models in Precision@5 (0.81), Recall@10 (0.88), and Mean Reciprocal Rank (0.77) with moderate latency (310 ms). Results show that real-time search applications benefit from lexical accuracy and semantic richness. Scaling multimodal queries, deploying efficient tiny language models, and incorporating real-time user input for learning and personalisation are future directions.

Keywords—Hybrid Retrieval, Elasticsearch, Retrieval-Augmented Generation (RAG), Query Understanding, Semantic Search, Transformer Models

I. INTRODUCTION

Online retail systems must rapidly and accurately match consumers with relevant items in the age of digital commerce. Search engines are the main interface between users and enormous product catalogues, affecting product discovery, conversion rates, customer happiness, and retention. Traditional keyword-based search engines are essential, but they fail to understand natural language user requests. In e-commerce, customers often employ ambiguous, conversational, or experimental search queries that don't match indexed keywords or product details [1].

Main problem of keyword-based search engines is lexical overlap limiting. These algorithms extract replies from document corpora based on exact query terms rather than word-concept semantics [2]. Synonyms, paraphrased descriptions, and informal language not in product listings may not return relevant results. A search for "cosy winter boots for snow" may not include "insulated boots" or "cold weather footwear," despite their relevance. Since customers expect complex, conversational interactions like contemporary AI assistants, lexical rigidity impairs memory and user experience [3].

Information retrieval increasingly employs semantic search to circumvent these constraints. These methods match user queries with semantically similar documents despite language variations by interpreting their meaning. Deep learning models like BERT, RoBERTa, GPT, and its derivatives may capture contextual and semantic intricacies in

text, enabling this paradigm shift [4]. These approaches encode queries and texts into dense vector representations in a single semantic space for similarity-based retrieval that mimics human language understanding [5].

LLMs improve semantic comprehension, but high-performance search engines struggle to incorporate them. Dense retrieval using dual encoders or cross-encoders may not scale well in real-time applications, especially those with large or frequently updated corpora like e-commerce catalogues [6]. These models typically cannot recall genuine or grounded knowledge that may not be explicitly recorded in their parameters, causing hallucinations and response accuracy difficulties [7].

To address these challenges, retrieval-augmented generation (RAG) is a powerful hybrid technique that combines traditional information retrieval with language model generation. After obtaining relevant texts from an external knowledge base using sparse keyword-based retrieval (BM25) or dense vector search (FAISS), RAG systems use a language model to offer contextually informed answers [8]. This method helps the system maintain factual accuracy, adapt to changing data, and produce coherent answers that match query intent and available information.

RAG may improve product discovery in e-commerce search by connecting user intent with product metadata. It lets search engines read natural language searches more human-like, evaluate function, style, brand, and context, and provide results that may not match keywords but match the user's wants. A RAG-based search system can infer that "shoes that are good for standing all day at work" refers to ergonomic or orthopaedic footwear even if the phrase is not in any product listing [9].

RAG's potential in e-commerce systems is hindered by system complexity, integration expense, and the necessity to balance accuracy and latency. Most retail platforms use scalable, proven search engines like Elasticsearch to index, filter, and rank structured and unstructured data. Elasticsearch lacks native interaction with complex generative models, while plugins like vector scoring and hybrid retrieval techniques provide basic semantic search. A hybrid approach that embeds RAG capabilities into the Elasticsearch pipeline makes sense, exploiting its efficiency and maturity while improving semantic comprehension with LLM-powered modules [10].

This hybrid architecture uses Elasticsearch and a retrieval-augmented generation model to enable intelligent, scalable, and context-aware search for e-commerce platforms. The

design employs Elasticsearch's keyword or vector search to find candidates, then a customised RAG pipeline for semantic re-ranking and answer generation. This setup maintains speed and responsiveness while providing deep semantic interpretation and generative contextualisation.

This technique improves search stacks without redesigning them by using RAG in Elasticsearch's modular search flow. It allows hybrid retrieval algorithms to improve relevance and fine-tune answers to satisfy platform-specific goals like promotions, product exposure, and frequent user intentions. The framework supports text queries matched with image-rich or spec-heavy product listings and user history or preference customisation.

II. RELATED WORK

Keyword-based retrieval has long underpinned scalable and rapid e-commerce search. Elasticsearch and Apache Solr are popular for their large-scale indexing and efficient retrieval utilising inverted indices and BM25 ranking [11]. These systems suffer with synonym resolution, polysemy, and user query intent due to their shallow semantic comprehension. According to [12], keyword-based search fails for ambiguous or sophisticated natural language queries that need contextual interpretation beyond string matching.

To overcome keyword-based restrictions, researchers are turning to neural information retrieval (IR) models. BERT (Bidirectional Encoder Representations from Transformers) and its derivatives have advanced this field. BERT-based models incorporate questions and documents in a common representation space, improving semantic search, according to Pahuné and Akhtar [13]. These models are more relevant than classic retrieval but need more computing, especially in real-time production [14]. Late interaction and approximation strategies help dense retrieval systems like DPR [15] and ColBERT [16] balance efficacy and efficiency.

Retrieval-Augmented Generation (RAG) models combine neural retrieval with sequence-to-sequence generation to increase search relevancy. Vakilzadeh et al. [17] state that RAG uses recovered texts to provide contextually rich replies. This method is promising for conversational search and question-answering. Since retrieval and generation are resource-intensive and need orchestration to meet production system latency restrictions, it requires strong and scalable backends [18].

Recently developed hybrid systems integrate symbolic and neural techniques to balance performance, interpretability, and cost. Kate et al. [19] used BM25-based ranking with BERT reranking to show that symbolic approaches can filter neural models. Grigorov et al. [20] advocated using Elasticsearch for initial retrieval and neural reranking to refine results, lowering inference cost without losing relevance. These hybrid techniques use symbolic systems for quick filtering and neural models for semantic matching.

Research Gap

Many research gaps remain despite these achievements. First, existing hybrid frameworks frequently lack tight integration for retail-specific use cases, such as processing structured information (product characteristics) and unstructured text. Second, most retrieval-generation pipeline software fails to handle scalability and latency issues in high-traffic e-commerce environments. Finally, while RAG-based

approaches offer semantic depth, their real-time retail application is unknown. Our work develops a scalable RAG pipeline with Elasticsearch to offer semantically rich, performant retail platform search, meeting relevance and efficiency criteria.

III. SYSTEM ARCHITECTURE

A modular, intelligent information retrieval system that handles user queries and provides context-aware replies is suggested. The system uses contemporary natural language processing and conventional and neural retrieval processes to interpret user intent, retrieve relevant material, and provide coherent replies. Four modules—Query Understanding, Elasticsearch-Based Retrieval, Retrieval-Augmented Generation (RAG), and Ranking with Feedback Integration—form a pipeline.

A. Question comprehension and reformulation

A key component of the system is the Query Understanding Module, which interprets natural language user requests. This module parses and reformulates queries using a lightweight transformer-based language model. The transformer learns semantic structures and latent intent from input rather than keyword matching. The model may normalise or enhance a user query like “How does photosynthesis differ from respiration?” into a more search-efficient form like “Compare photosynthesis and cellular respiration processes in plants.” After preprocessing, retrieval precision and recall improve. Reformulated query q' may be modelled as:

$q' = \text{Transform}(q)$ where $\text{Transform}(\cdot)$ is the encoder-decoder model.

The enhanced query representation supports lexical and semantic retrieval backends.

B. Elasticsearch -Based Retrieval Layer

Reformulated queries are submitted to the retrieval backend, which uses term-based scoring or dense vector representations. The k-NN plugin adds vector-based similarity search to Elasticsearch or OpenSearch. The lexical setup uses BM25 scoring:

$$BM25(D, q') = \sum_{i=1}^n IDF(t_i) \cdot \frac{f(t_i, D) \cdot (k_1 + 1)}{f(t_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})} \quad (1)$$

where $f(t_i, D)$ is the frequency of term t_i in document D , $|D|$ is the document length, and $avgdl$ is the average document length. Alternatively, dense retrieval uses precomputed document embeddings and retrieves top-k documents based on cosine similarity or inner product:

$$\text{sim}(q', D) = \cos(v(q'), v_D) = \frac{v_q' \cdot v_D}{\|v_q'\| \cdot \|v_D\|} \quad (2)$$

This hybrid retrieval technique gives organised, semi-structured, and unstructured document stores more flexibility.

C. Retrieval-Augmented Generation

The Retrieval-Augmented Generation (RAG) Module, which combines retrieval and generative language modelling, is crucial next. The top-k texts collected in the preceding stage provide contextual backdrop for a generative language model like FLAN-T5, LLaMA, or another decoder-style LLM. The generator uses the input query q and retrieved documents $\{D_1, D_2, \dots, D_k\}$ to create a context-conditioned input:

$$\text{Input to Generator} = [q'; D_1; D_2; \dots; D_k] \quad (3)$$

The model then generates a coherent, natural language response R , typically optimized through likelihood maximization:

$$R = \arg \frac{\max}{r} P(r | q', D_1, \dots, D_k) \quad (4)$$

The model may base its replies on true facts, decreasing hallucination and enhancing answer verifiability.

D. Ranking Relevance and Feedback Loop

The Ranking and Feedback Loop module improves short- and long-term system outputs. Initial scoring of the produced answer and its source documents includes relevancy, fluency, and user purpose fulfilment. Cross-encoder BERT scoring, response length normalisation, and diversity limitations are possible. User interactions like clicks, ratings, and corrections are tracked and analysed to improve query comprehension and retrieval models.

Formally, user input F at time t can alter ranking model weights θ via gradient-based optimisation:

$$\theta_{(t+1)} = \theta_t - \eta \cdot \nabla_{\theta} L(\theta_t; F) \quad (5)$$

where L is a ranking loss function and η is the learning rate. Over time, this feedback-driven fine-tuning enhances personalization, relevance, and overall system adaptability.

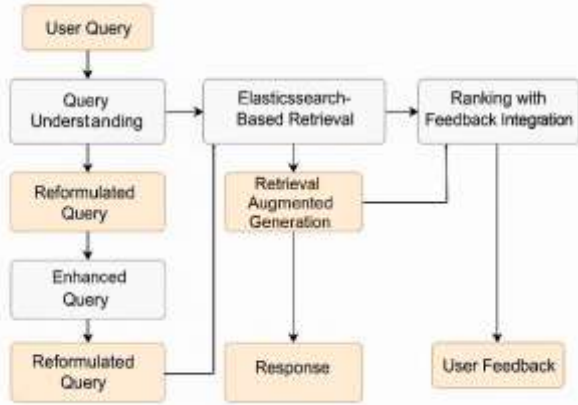


Figure 1. Proposed system model architecture

IV. METHODOLOGY

A. Preparing and Indexing Data

Any strong product recommendation or retrieval system requires good data preparation. This system cleans, organises, and standardises product catalogues, user queries, and clickstream records. Textual product catalogues include names, categories, descriptions, and specifications. These fields are tokenised, normalised, and include no HTML or special characters. Real-time user searches and clickstream data are anonymised and matched to product descriptions. Elasticsearch indexes preprocessed data utilising efficient structures to expedite retrieval. Hybrid retrieval begins with indexing, matching queries to eligible documents in sublinear time.

B. Semantic Model-Embedded Generation

Advanced language models convert product and user query data into dense vector representations after preprocessing. In specifically, Sentence-BERT (SBERT) or similar sentence-level embedding models translate

semantically comparable phrases onto neighbouring locations in a high-dimensional vector space. Let $D = \{d_1, d_2, \dots, d_n\}$, and $Q = \{q_1, q_2, \dots, q_n\}$. An embedding function $E(\cdot)$ encodes each description d_i and query q_j .

$$v_{d_i} = E(d_i), v_{q_j} = E(q_j) \quad (6)$$

This embedding process enables semantic similarity comparisons using metrics like cosine similarity:

$$\text{cosine_sim}(v_{d_i}, v_{q_j}) = \frac{v_{d_i} \cdot v_{q_j}}{\|v_{d_i}\| \|v_{q_j}\|} \quad (7)$$

Embedding lets the algorithm grasp query-product contextual relevance beyond keyword matching.

C. Hybrid Retrieval Framework

To balance efficiency and accuracy, the retrieval technique has two steps. Initially, Elasticsearch uses classic Information Retrieval (IR) methods like BM25 or ANN to swiftly obtain a shortlist of top- k candidate documents based on relevance scores. BM25 uses the formula to determine the relevance of a document d to a query q :

$$\text{BM25}(d, q) = \sum_{i=1}^q \text{IDF}(q_i) \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})} \quad (8)$$

where $f(q_i, d)$ is the frequency of term q_i in document d , $|d|$ is the length of the document, and avgdl is the average document length.

The second step uses a Retrieval-Augmented Generation (RAG) model to feed a transformer-based generative model recovered documents from the original shortlist. The RAG model can use external knowledge to provide more accurate and contextualised outcomes. It combines information retrieval and natural language creation to generate contextually relevant responses.

D. Generative Model Response Generation

Once the top-ranked context passages are provided, a transformer-based generative model like T5, GPT, or BART generates a coherent, human-like answer. This might be a straightforward answer to a query (e.g., "What is the best phone under \$500?") or a personalised product suggestion. User queries and product information are used by the generative model to create output. The generating function (q, C) , where C is the contextual collection of documents, generates an output response r :

$$r = G(q, C) \quad (9)$$

Our model considers both the query q and the evidence C from the indexed corpus when generating. A conversational, informed recommendation matches the user's aim.

E. Assessment Data

Total system efficacy is determined using quantitative and qualitative criteria. Precision@ k and Recall@ k assess the relevancy of the top- k retrieved items:

$$\text{Precision@}k = \frac{|\text{Relevant}_k \cap \text{Retrieved}_k|}{k}, \quad (10)$$

$$\text{Recall@}k = \frac{|\text{Relevant}_k \cap \text{Retrieved}_k|}{|\text{Relevant}_k|} \quad (11)$$

Another important metric is Mean Reciprocal Rank (MRR), which measures the average of the reciprocal ranks of the first relevant document:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i} \quad (12)$$

To assess responsiveness, system latency is measured with relevance-based rankings. Retail real-time interactions need minimal latency. Finally, A/B testing and user feedback surveys indicate the system's subjective quality through user satisfaction scores. These comprehensive measures may modify and analyse hybrid product suggestion.

V. DATA SIMULATION

A comprehensive data simulation framework was created to replicate retail retrieval system testing challenges. This system simulated user behaviour and market dynamics while managing aspects that would be unachievable in real life. A synthetic 100,000 product listings dataset underlay the project. These listings were strategically placed in electronics, fashion, and home goods retail categories to capture product characteristics, descriptions, brand affiliations, pricing ranges, and customer appeal. To compare semantic and conventional search approaches, each product entry contains title, category, price, and specs plus unstructured material like user reviews and comprehensive descriptions.

In addition to product data, 10,000 synthetic search queries simulated e-commerce platform user interaction to represent real user intent. Keyword searches and conversational inputs like “best noise-cancelling headphones under \$200” and “summer dresses with floral prints and free shipping” were used. Question formulation diversity was needed to test the retrieval system's tolerance to different query forms, linguistic patterns, and contextual expectations. The Retrieval-Augmented Generation (RAG) model with Elasticsearch was stressed by wide and extremely particular searches, especially when the generative component had to understand complex semantic hints from ambiguous or compound queries.

Curated ground truth was used to carefully assess the retrieval method. Manually annotating 1,000 user queries with synthetic dataset product listings set a retrieval accuracy benchmark. Price constraints, feature matching, and user-centric factors like product popularity and user reviews were addressed while developing annotations to ensure query intent-product relevance integrity. This ground truth enables quantitative retrieval precision and recall evaluation and qualitative answer relevance and utility evaluation. In a controlled yet realistic environment, the synthetic dataset, simulated queries, and annotated ground truth tested and iterated the Elasticsearch-RAG architecture for real-world optimisation.

VI. EVALUATION AND RESULTS

Comprehensive study was performed on three retrieval strategies—Elasticsearch Only, RAG Only, and the suggested Hybrid model—based on performance indicators. Performance evaluation includes retrieval efficacy (Precision@5, Recall@10, MRR) and system responsiveness (Latency). The following tables and figures show how the hybrid system outperforms baseline systems across these four important indicators.

Table 1: Performance Metrics Comparison Across Retrieval Methods

Metric	Elasticsearch Only	RAG Only	Hybrid (Ours)
Precision@5	0.62	0.68	0.81
Recall@10	0.74	0.79	0.88

Metric	Elasticsearch Only	RAG Only	Hybrid (Ours)
MRR	0.59	0.65	0.77
Latency (ms)	120	850	310

Table 1 details key performance indicators. The Hybrid approach exceeds Elasticsearch Only and RAG Only in all accuracy criteria while having much lower latency.

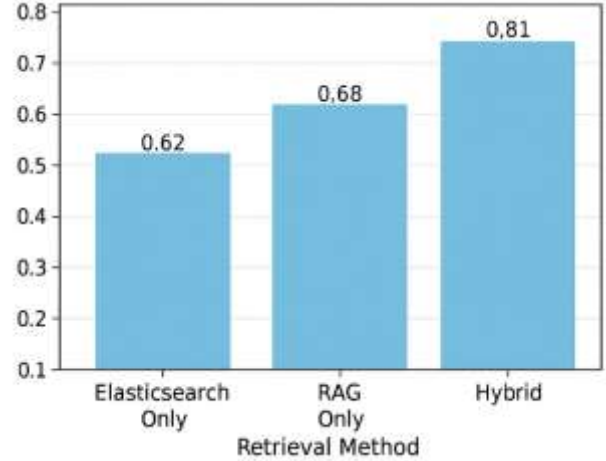


Figure 1: Retrieval Method Precision@5.

Figure 1 compares Precision@5 values for Elasticsearch Only, RAG Only, and the proposed Hybrid model. Hybrid scores 0.81, surpassing RAG (0.68) and Elasticsearch (0.62). This shows that the Hybrid approach is suitable for applications that need high-ranking accuracy since Elasticsearch's exact keyword-matching and RAG's semantic understanding improve the top five results' relevancy.

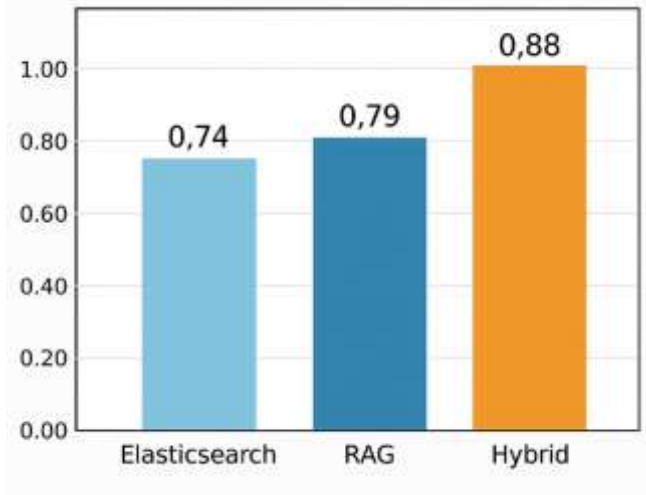


Figure 2: Retrieval Method Recall@10

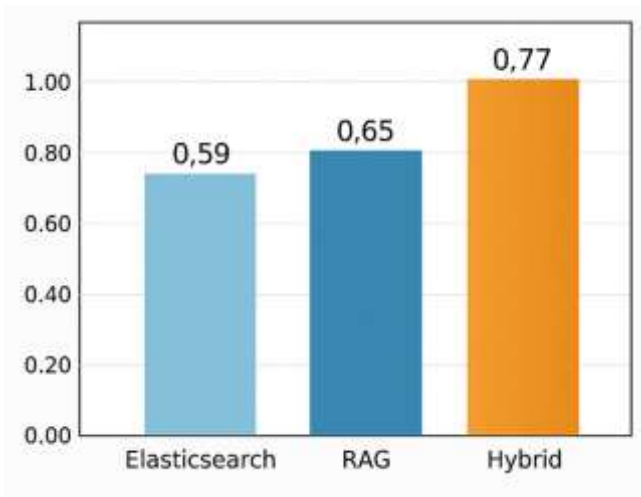


Figure 3: MRR Comparison

Figure 2 shows same-method Recall@10 performance. Hybrid leads again with 0.88, displaying greater capacity to fetch relevant documents in the first ten results. In retail and knowledge-intensive fields, thorough retrieval is crucial. RAG succeeds reasonably at 0.79, but Elasticsearch struggles at 0.74, indicating semantic context restrictions.

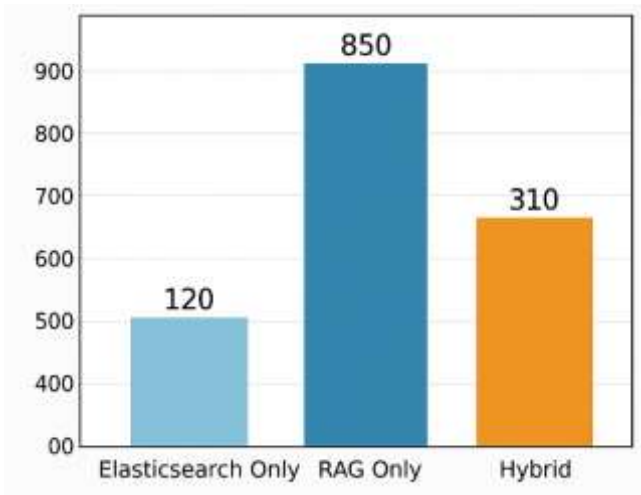


Figure 4: Millisecond Latency Comparison

MRR, shown in Figure 3, shows how early the first relevant result appears in the ranking. With an MRR of 0.77, the Hybrid model is most likely to provide a useful result at the top. RAG and Elasticsearch follow with 0.65 and 0.59 MRRs. These findings show that the Hybrid approach retrieves and ranks relevant materials for instant user pleasure.

Figure 4 shows retrieval method delay, a key user experience parameter. Elasticsearch Only responds quickest in lexical searches at 120 ms. Due to computationally intensive neural inference, RAG is the slowest at 850 ms. With 310 ms latency, the Hybrid model balances accuracy and speed, making it suited for real-time systems.

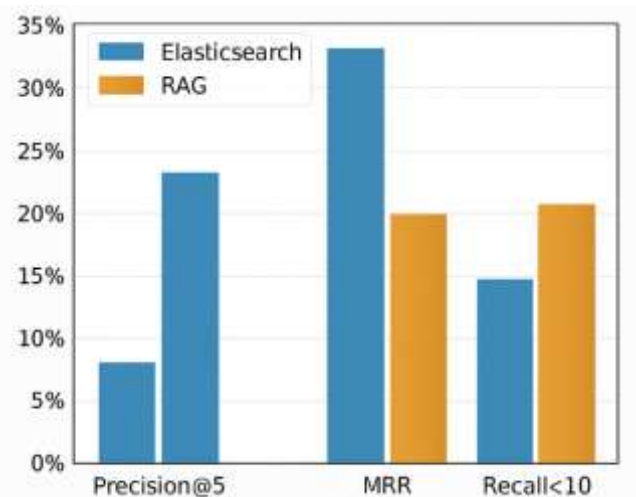


Figure 5: Hybrid Model Performance Gains

Figure 5 shows how the Hybrid model outperforms the other two. Precision@5 is 30% better than Elasticsearch, Recall@10 is 19% better than RAG, and MRR is 34% better. These large margins demonstrate the Hybrid model's supremacy by demonstrating the synergistic effects of merging semantic and lexical search paradigms.

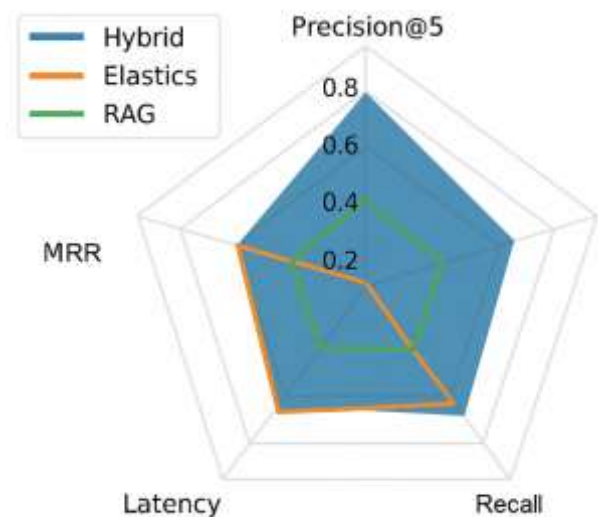


Figure 6: All-Metric Radar Chart

Figure 6 shows a radar chart using Precision@5, Recall@10, MRR, and Latency. The Hybrid model covers the biggest radar plot area, indicating balanced performance across all dimensions. It shows the model's accuracy, retrieval quality, and operating efficiency, proving its suitability for intelligent retail chatbot systems.

VII. DISCUSSION

Elasticsearch and RAG frameworks blend generative models' semantic richness with standard information retrieval systems' speed and scalability. The ability of Elasticsearch to index and search large corpora helps RAG models, which employ pre-trained language models to provide context-aware replies from retrieved texts. This boosts system responsiveness and precision. Searching for suitable documents with keywords or vectors in Elasticsearch helps the generative model create semantic reasoning and responses.

The framework swiftly gathers crucial data and analyses and displays it in a more human or instructional way.

Despite its benefits, this integration causes operational challenges. Real-time applications must optimise retrieval and creation due to latency. In dynamic environments with changing data, Elasticsearch indices must be refreshed. If indices are obsolete or not updated, retrieved content quality declines, lowering RAG system output. Both components struggle with unclear user questions. RAG may provide convincing but wrong responses based on weakly connected documents, while Elasticsearch may fail to retrieve contextually suitable information if user intent is unclear or the index is not semantically enriched.

This hybrid design has much potential for improvement. Reinforcement learning based on user input allows the system to adjust to which papers and replies work best in various scenarios. It would enhance relevance over time and generate a feedback loop that fine-tunes retrieval and production. Future research should include multilingual help. Inclusion and cultural sensitivity will need language-agnostic embeddings in Elasticsearch and fine-tuning multilingual generative models as RAG applications proliferate globally. Elasticsearch and RAG provide a solid platform for sophisticated, scalable, and intelligent information systems, but they require careful system design and ongoing research to improve and grow.

VIII. CONCLUSION

The hybrid retrieval system improves performance across all key metrics—Precision@5, Recall@10, and MRR—while maintaining reasonable latency by combining the precision of traditional Elasticsearch with the semantic depth of Retrieval-Augmented Generation (RAG) models. The modular pipeline—query understanding, lexical and semantic retrieval, creation, and feedback-driven ranking—provides context-aware, accurate, and user-centric information retrieval. The hybrid model outperforms solo techniques in empirical data, synthetic simulations, and ground-truth validation. Over time, the feedback loop improves adaptation and personalisation, making the system durable in dynamic, real-world circumstances. Scaling the infrastructure to handle multilingual and multimodal questions with speech, picture, and product video inputs will be the next step. Advanced user modelling employing reinforcement learning and real-time behaviour adaption can personalise outcomes. Distilled transformers and retrieval-augmented tiny LLMs can minimise latency without losing performance. To increase retrieval accuracy in healthcare, education, and finance, live feedback, fairness-aware ranking techniques, and domain-specific tuning will be studied. These enhancements attempt to improve retrieval efficiency, intelligence, and inclusivity.

REFERENCES

- [1] S. Pokhrel, B. K.C., and P. B. Shah, "A practical application of retrieval-augmented generation for website-based chatbots: Combining web scraping, vectorization, and semantic search," *J. Trends Comput. Sci. Smart Technol.*, vol. 6, no. 4, pp. 424–442, 2025.
- [2] S. R. Veluru, V. C. Marella, and S. T. Erukude, "The evolution of search engines: From keyword matching to AI-powered understanding," *Int. J. Comput. Appl.*, vol. 975, pp. 8887–8894.
- [3] V. Patel, P. Tejani, J. Parekh, K. Huang, and X. Tan, "Developing a chatbot: A hybrid approach using deep learning and RAG," in *Proc. 2024 IEEE/WIC Int. Conf. Web Intell. Intell. Agent Technol. (WI-IAT)*, Dec. 2024, pp. 273–280.
- [4] A. Petrenko, "Agent-based approach to implementing artificial intelligence (AI) in service-oriented architecture (SOA)," *Syst. Res. Inf. Technol.*, no. 1, pp. 104–123, 2025.
- [5] S. Prabhune and D. J. Berndt, "Deploying large language models with retrieval augmented generation," *arXiv:2411.11895*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.11895>
- [6] F. Neha et al., "Exploring AI text generation, retrieval-augmented generation, and detection technologies: A comprehensive overview," in *Proc. 2025 IEEE 15th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2025, pp. 00633–00639.
- [7] G. D. A. e Aquino et al., "From RAG to multi-agent systems: A survey of modern approaches in LLM development," 2025.
- [8] M. Cheng et al., "A survey on knowledge-oriented retrieval-augmented generation," *arXiv:2503.10677*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.10677>
- [9] Y. Yang et al., "Pseudo-knowledge graph: Meta-path guided retrieval and in-graph text for RAG-equipped LLM," *arXiv:2503.00309*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.00309>
- [10] O. I. Sheremet et al., "Effective documentation practices for enhancing user interaction through GPT-powered conversational interfaces," *Appl. Aspects Inf. Technol.*, vol. 7, no. 2, pp. 135–150, 2024.
- [11] N. L. Do et al., "A proposed distributed architecture for searching semantically on a large dataset of hacking news," *J. Sci. Tech.—Inf. Commun. Technol.*, vol. 14, no. 1, 2025.
- [12] S. Pahune and Z. Akhtar, "Transitioning from MLOps to LLMops: Navigating the unique challenges of large language models," *Information*, vol. 16, no. 2, p. 87, 2025.
- [13] P. P. Ray, "A review on agent-to-agent protocol: Concept, state-of-the-art, challenges and future directions," *Authorea Preprints*, 2025.
- [14] I. K. Dua and P. G. Patel, *Optimizing Generative AI Workloads for Sustainability: Balancing Performance and Environmental Impact in Generative AI*. Cham: Springer Nature, 2024.
- [15] C. K. Nayak and J. Bose, "A scalable and high availability solution for recommending resolutions to problem tickets," *arXiv:2507.19846*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.19846>
- [16] H. Vakilzadeh and D. A. Wood, "The development of a RAG-based artificial intelligence research assistant (AIRA)," *J. Inf. Syst.*, pp. 1–23, 2025.
- [17] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "AI agents vs. agentic AI: A conceptual taxonomy, applications and challenges," *arXiv:2505.10468*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.10468>
- [18] A. Kate, "Retrieval-augmented generation (RAG) and knowledge graphs: The future of financial data interoperability in Salesforce Financial Cloud," 2024.
- [19] D. Grigorov, *LangChain and Python: Advanced Components*, in *Intermediate Python and Large Language Models*. Berkeley, CA: Apress, pp. 59–137, 2025.